

OS CIE-1

System Detective

Linux System Inspection and Memory Locality Analysis

Jayachandra Vandanapu

TY IT – Roll No. 33284

Pune Institute of Computer Technology

1. Objective

The aim of this activity was to inspect a Linux system, perform a memory locality experiment, and display the results using a dashboard.

2. Experimental Environment

The experiments were performed inside WSL2 using Ubuntu 24.04.2 LTS.

Property	Value
Environment	WSL2
Operating System	Ubuntu 24.04.2 LTS
Architecture	x86_64
Kernel	5.15.167.4-microsoft-standard-WSL2
Processor	11th Gen Intel Core i5-1135G7
Logical CPUs	8
Cores / Threads per core	4 / 2
Compiler	GCC 13.3.0
Node.js	18.19.1

Table 1: Experimental environment and development tools

3. System Inspection

The following Linux commands were used to inspect the system:

- `lscpu` – CPU architecture, cores, threads and cache information.
- `free -h` – memory and swap information.
- `top` – current process and CPU activity.
- `ps -eLf` – processes and lightweight process information.
- `strace -c` – summary of system calls made by a command.

The `lscpu` output showed an x86_64 environment with 8 logical CPUs, 4 cores and 2 threads per core.

```

vanda@JAY-16CHANDRA:~/system-detective$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 8
On-line CPU(s) list:    0-7
Vendor ID:              GenuineIntel
Model name:             11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz
CPU family:             6
Model:                  140
Thread(s) per core:     2
Core(s) per socket:     4
Socket(s):              1
Stepping:               1
BogoMIPS:               4838.39
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse s
se2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon rep_good nopl xtopology tsc_r
eliable nonstop_tsc cpuid pni pclmulqdq vmx ssse3 fma cx16 pdcm pcid sse4_1 sse4_2 x2apic movb
e popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch inv
pcid_single ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow vnmi ept vpid ept_ad fsgsbase tsc_ad
just bmi1 avx2 smep bmi2 erms invpcid avx512f avx512dq rdseed adx smap avx512ifma clflushopt c
lwb avx512cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsaves avx512vbmi umip avx512_vb
mi2 gfni vaes vpclmulqdq avx512_vnni avx512_bitalg avx512_vpopcntdq rdpid movdiri movdir64b fs
rm avx512_vp2intersect md_clear flush_l1d arch_capabilities

Virtualization features:
Virtualization:         VT-x
Hypervisor vendor:      Microsoft
Virtualization type:    full
Caches (sum of all):
L1d:                    192 KiB (4 instances)
L1i:                    128 KiB (4 instances)
L2:                     5 MiB (4 instances)
L3:                     8 MiB (1 instance)
Vulnerabilities:
Gather data sampling:    Unknown: Dependent on hypervisor status
Itlb multihit:           Not affected
L1tf:                    Not affected
Mds:                     Not affected
Meltdown:                Not affected
Mmio stale data:         Not affected
Reg file data sampling:  Not affected
Retbleed:                Mitigation; Enhanced IBRS
Spec rstack overflow:    Not affected
Spec store bypass:       Mitigation; Speculative Store Bypass disabled via prctl and seccomp
Spectre v1:              Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:              Mitigation; Enhanced / Automatic IBRS; IBPB conditional; RSB filling; PBRSE-eIBRS SW sequence;
                        BHI SW loop, KVM SW loop
Srbds:                   Not affected
Tsx async abort:         Not affected

```

Figure 1: CPU information obtained using `lscpu`.

The memory information was collected using `free -h`.

```

vanda@JAY-16CHANDRA:~/system-detective$ free -h
              total        used        free      shared  buff/cache   available
Mem:          3.7Gi        759Mi        1.2Gi        3.2Mi        2.0Gi        3.0Gi
Swap:         1.0Gi           0B          1.0Gi

```

Figure 2: Memory information obtained using `free -h`.

The `top` command provided a snapshot of CPU activity, memory usage and process states. The `ps -eLf` command showed processes and their lightweight processes (threads). I also used `strace -c` to examine the system calls made by a command. These outputs were saved in the project for reference.

```
vanda@JAY-16CHANDRA:~/system-detective$ top -b -n 1
top - 16:07:25 up 16 min, 1 user, load average: 1.03, 0.86, 0.42
Tasks: 32 total, 1 running, 31 sleeping, 0 stopped, 0 zombie
%Cpu(s): 1.2 us, 2.4 sy, 0.0 ni, 95.2 id, 0.0 wa, 0.0 hi, 1.2 si, 0.0 st
MiB Mem : 3818.0 total, 1530.9 free, 672.2 used, 1878.1 buff/cache
MiB Swap: 1024.0 total, 1024.0 free, 0.0 used. 3145.9 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
    1 root        20   0   21980  13372  9612  S   0.0   0.3   0:03.48 systemd
    2 root        20   0   2776   1924  1796  S   0.0   0.0   0:00.00 init-systemd(Ub
    6 root        20   0   2776   132   132  S   0.0   0.0   0:00.00 init
   158 root        20   0   4236   2752  2524  S   0.0   0.1   0:00.49 cron
   159 message+    20   0   9920   5440  4556  S   0.0   0.1   0:00.78 dbus-daemon
   169 root        20   0   17976  8276  7256  S   0.0   0.2   0:00.22 systemd-logind
   172 root        20   0 1755840 16336  9768  S   0.0   0.4   0:00.27 wsl-pro-service
   178 root        20   0   3160   1152  1060  S   0.0   0.0   0:00.00agetty
   182 root        20   0   3116   1068   984  S   0.0   0.0   0:00.00agetty
   194 root        20   0 107024 22304 12964  S   0.0   0.6   0:00.12 unattended-upgr
   320 root        20   0   2780    208    80  S   0.0   0.0   0:00.00 SessionLeader
   321 root        20   0   2780    212    80  S   0.0   0.0   0:00.03 Relay(322)
   322 vanda        20   0   6072  5304  3572  S   0.0   0.1   0:00.09 bash
   323 root        20   0   6688  4808  4028  S   0.0   0.1   0:00.01 login
   412 vanda        20   0 20044 10924  9184  S   0.0   0.3   0:00.34 systemd
   413 vanda        20   0  21144  1724    0  S   0.0   0.0   0:00.00 (sd-pam)
   428 vanda        20   0   6072  5196  3556  S   0.0   0.1   0:00.01 bash
  2048 root        19  -1  50340 15852 14712  S   0.0   0.4   0:00.16 systemd-journal
  2108 systemd+    20   0  91024  6560  5708  S   0.0   0.2   0:00.04 systemd-timesyn
  2169 root        20   0  24108  6292  5000  S   0.0   0.2   0:00.33 systemd-udev
  2219 systemd+    20   0  21456 11852  9656  S   0.0   0.3   0:00.06 systemd-resolve
  3356 polkitd     20   0 308164 10148  7240  S   0.0   0.3   0:00.13 polkitd
 11615 root        20   0   2804   1052   964  S   0.0   0.0   0:00.00 apt.systemd.dai
 13404 root        20   0 370084 20652 17740  S   0.0   0.5   0:00.10 packagekitd
 14111 syslog      20   0 222512  5160  4568  S   0.0   0.1   0:00.03 rsyslogd
 18130 root        20   0   2804   1936  1780  S   0.0   0.0   0:00.00 apt.systemd.dai
 18174 root        20   0  15156 10132  7820  S   0.0   0.3   0:00.02 apt-get
 18183 _apt         20   0  20300  9684  8632  S   0.0   0.2   0:00.03 http
 18184 _apt         20   0  20300  9692  8640  S   0.0   0.2   0:00.00 http
 18235 _apt         20   0  13096  6956  6352  S   0.0   0.2   0:00.00 gpgv
 18306 _apt         20   0  21480  8364  7204  S   0.0   0.2   0:00.02 store
 18307 vanda        20   0   9340   5372  3260  R   0.0   0.1   0:00.00 top
```

Figure 3: Process and CPU activity observed using top.

```
vanda@JAY-16CHANDRA:~/system-detective$ ps -elf
F S UID      PID  PPID  C PRI  NI ADDR SZ WCHAN  STIME TTY          TIME CMD
4 S root        1      0   0  80  0 - 5495 - 15:50 ?    00:00:03 /usr/lib/systemd/systemd --system --deserialize=50
4 S root        2      1   0  80  0 - 694 - 15:50 ?    00:00:00 /init
0 S root        6      2   0  80  0 - 694 - 15:50 ?    00:00:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
4 S root       158      1   0  80  0 - 1859 - 15:50 ?    00:00:00 /usr/sbin/cron -f -P
4 S message+   159      1   0  80  0 - 2480 - 15:50 ?    00:00:00 dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
4 S root       169      1   0  80  0 - 4494 - 15:50 ?    00:00:00 /usr/lib/systemd/systemd-logind
4 S root       172      1   0  80  0 - 438960 - 15:50 ?    00:00:00 /usr/libexec/wsl-pro-service -vv
4 S root       178      1   0  80  0 - 799 - 15:50 hvci 00:00:00 /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 vt220
4 S root       182      1   0  80  0 - 779 - 15:50 ttyl 00:00:00 /sbin/agetty -o -p -- \u --noclear - linux
4 S root       194      1   0  80  0 - 26756 - 15:50 ?    00:00:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
5 S root       320      2   0  80  0 - 695 - 15:50 ?    00:00:00 /init
5 S root       321     320   0  80  0 - 695 - 15:50 ?    00:00:00 /init
4 S vanda       322     321   0  80  0 - 1518 do_wai 15:50 pts/0 00:00:00 -bash
4 S root       323      2   0  80  0 - 1672 - 15:50 pts/1 00:00:00 /bin/login -f
4 S vanda       412      1   0  80  0 - 5811 - 15:50 ?    00:00:00 /usr/lib/systemd/systemd --user --deserialize=8
5 S vanda       413     412   0  80  0 - 5286 - 15:50 ?    00:00:00 (sd-pam)
4 S vanda       428     323   0  80  0 - 1518 core_s 15:50 pts/1 00:00:00 -bash
4 S root      2048      1   0  79 -1 - 32585 - 16:03 ?    00:00:00 /usr/lib/systemd/systemd-journald
4 S systemd+   2108      1   0  80  0 - 22756 - 16:03 ?    00:00:00 /usr/lib/systemd/systemd-timesyncd
4 S root      2169      1   0  80  0 - 6827 - 16:03 ?    00:00:00 /usr/lib/systemd/systemd-udev
4 S systemd+   2219      1   0  80  0 - 5364 - 16:03 ?    00:00:00 /usr/lib/systemd/systemd-resolved
4 S polkitd    3356      1   0  80  0 - 97841 - 16:04 ?    00:00:00 /usr/lib/polkit-1/polkitd --no-debug
4 S root     13404      1   0  80  0 - 92521 - 16:05 ?    00:00:00 /usr/libexec/packagekitd
4 S syslog    14111      1   0  80  0 - 55628 - 16:05 ?    00:00:00 /usr/sbin/rsyslogd -n -iNONE
0 R vanda     18307     322   0  80  0 - 2897 - 16:08 pts/0 00:00:00 ps -elf
```

Figure 4: Process and thread information obtained using ps -elf.

```
vanda@JAY-16CHANDRA:~/system-detective$ strace -c ls 2>&1
```

README.md	ai_log.md	dashboard	day1_notes.md	day2_notes.md	day3_notes.md	outputs	screenshots
% time	seconds	usecs/call	calls	errors	syscall		
18.17	0.000476	14	34	13	openat		
16.84	0.000441	14	30		mmap		
16.76	0.000439	219	2		getdents64		
11.95	0.000313	13	23		close		
11.11	0.000291	13	22		fstat		
6.34	0.000166	166	1		execve		
4.85	0.000127	18	7		read		
2.79	0.000073	14	5		mprotect		
1.45	0.000038	12	3		brk		
1.22	0.000032	16	2	2	access		
1.18	0.000031	31	1		write		
1.18	0.000031	15	2	2	statfs		
1.07	0.000028	14	2		ioctl		
1.03	0.000027	13	2		pread64		
0.73	0.000019	19	1		munmap		
0.50	0.000013	13	1		arch_prctl		
0.50	0.000013	13	1		futex		
0.50	0.000013	13	1		getrandom		
0.46	0.000012	12	1		set_tid_address		
0.46	0.000012	12	1		set_robust_list		
0.46	0.000012	12	1		prlimit64		
0.46	0.000012	12	1		rseq		
100.00	0.002619	18	144	17	total		

Figure 5: System-call summary obtained using `strace -c ls`.

4. Memory Locality Experiment

A 4096×4096 two-dimensional array of `double` values was used for the experiment. Each element occupies 8 bytes, giving an array size of approximately 128 MiB.

Two traversal methods were compared:

- **Row-major traversal:** rows are traversed from left to right before moving to the next row.
- **Column-major traversal:** columns are traversed from top to bottom before moving to the next column.

The benchmark was initialized with values of 1.0 and each traversal calculated a sum to ensure that the array was accessed. The program measured execution time using a monotonic clock and repeated each traversal three times. It also checked that both traversal methods produced matching sums.

The program was compiled and executed using:

```
gcc -O2 locality.c -o locality_c
./locality_c
```

The program was written in C and compiled using GCC with the `-O2` optimization level.

C stores two-dimensional arrays in row-major order. Therefore, row-major traversal accesses consecutive memory locations more frequently. Column-major traversal accesses elements that are farther apart in memory, resulting in poorer spatial locality.

5. Experimental Results

The locality experiment was executed three times. The following values are from the latest recorded benchmark run.

Run	Row-major (s)	Column-major (s)	Ratio
1	0.022200	0.252546	11.38×
2	0.020004	0.251715	12.58×
3	0.019275	0.254740	13.22×
Mean	0.020493	0.253000	12.35×

Table 2: Memory locality benchmark results

In all three runs, row-major traversal was faster than column-major traversal. The mean column-major execution time was approximately 12.35 times the mean row-major execution time.

The exact times changed slightly between runs, but the same overall pattern was observed.

```
vanda@JAY-16CHANDRA:~/system-detective/locality$ ./locality_c
Array size: 4096 x 4096
Total elements: 16777216
Element size: 8 bytes
Array size: 128.00 MiB

run,row_major_s,column_major_s,ratio
1,0.022200,0.252546,11.38
2,0.020004,0.251715,12.58
3,0.019275,0.254740,13.22

Mean row-major time: 0.020493 seconds
Mean column-major time: 0.253000 seconds
Mean ratio: 12.35x

Results saved to results.csv
```

Figure 6: Locality experiment results.

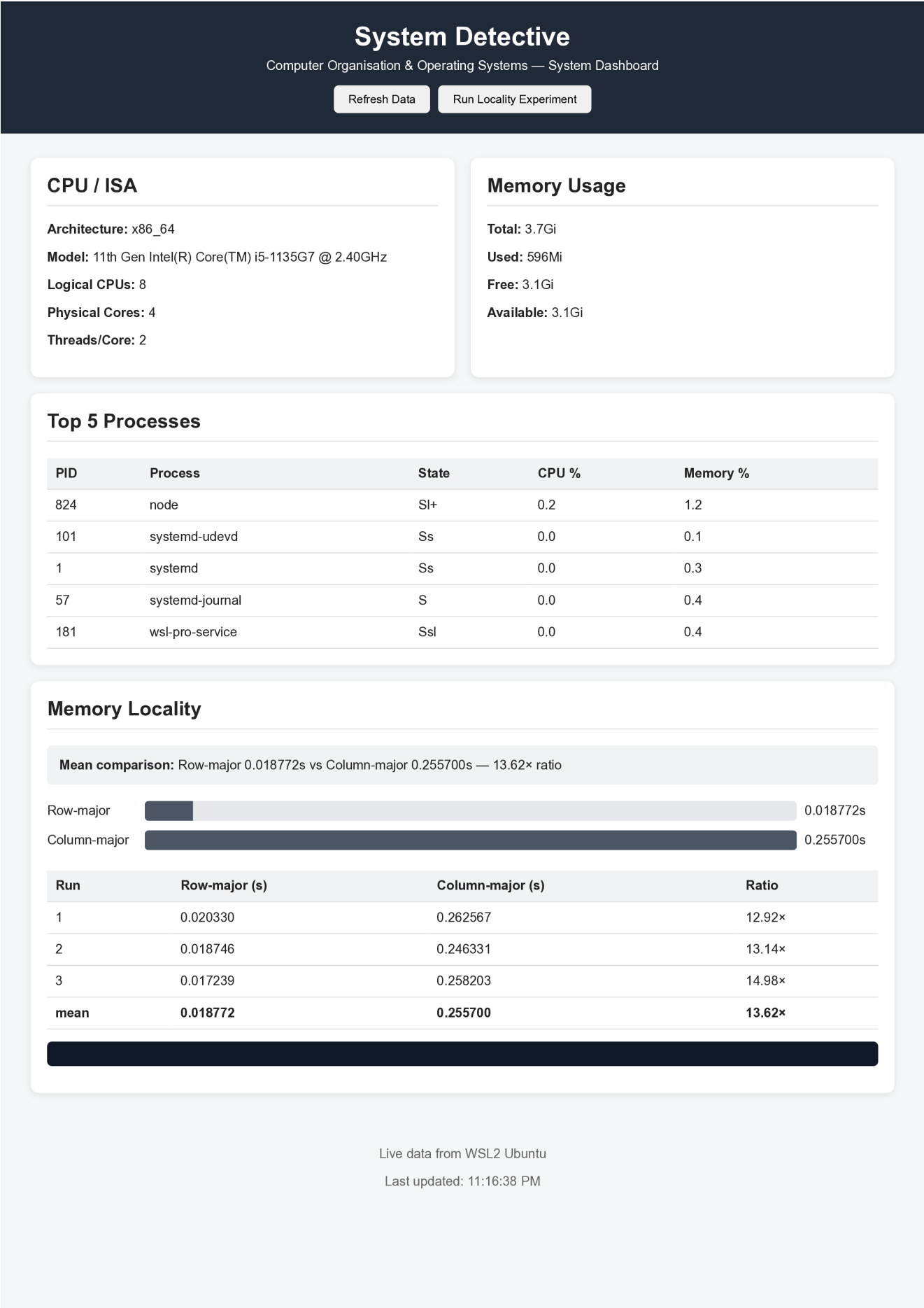
The benchmark was repeated on the same WSL2 environment using the same array dimensions and compiler optimization level. Although the measured times differed between executions, row-major traversal remained faster in the recorded runs. The results were saved in CSV format to allow comparison and reuse by the dashboard.

6. System Dashboard

A simple dashboard was developed using HTML, CSS, JavaScript and Node.js. The dashboard displays CPU information, memory usage, the top five processes and locality results.

The backend obtains system information from Linux commands and reads the locality benchmark results. It can also run the locality experiment again.

The dashboard provides a way to view the collected information in one place. Its system-data endpoint retrieves current information, while the locality endpoint can compile and run the benchmark again. The displayed process and memory values represent the system at the time of the request.



7. AI Verification and Reproducibility

I used AI during the project for debugging, explanations and help with the dashboard. System-specific results were checked by running the commands on my own WSL2 system.

One AI-generated prediction was also tested experimentally. The prediction stated that the process with the highest CPU usage would have a CPU value greater than 0.0%. The following command was executed:

```
ps -eo pid,comm,stat,%cpu,%mem --sort=-%cpu | head -n 6
```

The relevant part of the recorded output was:

```
PID COMMAND STAT %CPU %MEM
1 systemd Ss 0.0 0.3
93 systemd-udevd Ss 0.0 0.1
289 Relay(290) S 0.0 0.0
51 systemd-journal S<s 0.0 0.4
196 unattended-upgr Ssl 0.0 0.5
```

All five listed processes showed 0.0% CPU usage in this snapshot. Therefore, the prediction was incorrect for this particular observation. The recorded output showed 0.0% CPU usage for the listed processes, disproving the prediction. This showed that the AI prediction did not match the actual system output.

8. Project Files and Reproducibility

The project contains the C benchmark source code, CSV results, saved Linux command outputs, dashboard files and labelled screenshots. The benchmark can be compiled and run from the `locality` directory, while the dashboard can be started from the `dashboard` directory using Node.js. These files provide a record of the commands, implementation and results used in this activity.

9. Conclusion

This activity helped me understand Linux system information, processes, system calls and memory locality. The locality experiment showed a clear difference between row-major and column-major access. I also used the dashboard to bring the collected system and benchmark information together in one place.